CHAPTER 2

Parallel Computer Architecture

Parallel and Distributed Computing

Where work runs

Instructions, lanes, cores, and nodes

Where data lives

Caches, memory, and accelerators

What movement costs

Latency, bandwidth, and communication

LEARNING OBJECTIVES

What you will be able to explain

01 Recognize parallelism

Distinguish pipelines, SIMD, physical cores, hardware threads, and cluster processes.

02 Calculate memory costs

Work out cache capacity, average access time, bandwidth limits, and NUMA latency.

03 Reason about shared data

Explain coherence, consistency, synchronization, and false sharing.

04 Choose an architecture

Compare shared memory, clusters, and accelerators for a workload.

The same core count can perform differently

Question	Favorable pattern	Costly pattern
Where does data reside?	Near the core that uses it	Across a socket or network link
How is it accessed?	Contiguous access and reuse	Irregular access and little reuse
Who writes each region?	One clear owner	Frequent ownership transfers

Performance depends on data placement and movement as well as computation.

Parallelism appears at several levels

Level	Mechanism	Example
Instruction	Pipeline; superscalar issue	Overlap independent instructions
Data	SIMD or vector lanes	Add several array elements at once
Thread	Cores and hardware threads	OpenMP or Pthreads workers
Process	Separate address spaces / nodes	MPI ranks exchange messages
Accelerator	CPU with GPU or other device	Offload a matrix kernel

One application can combine MPI + threads + SIMD.

Pipelining overlaps instruction stages

Cycle →

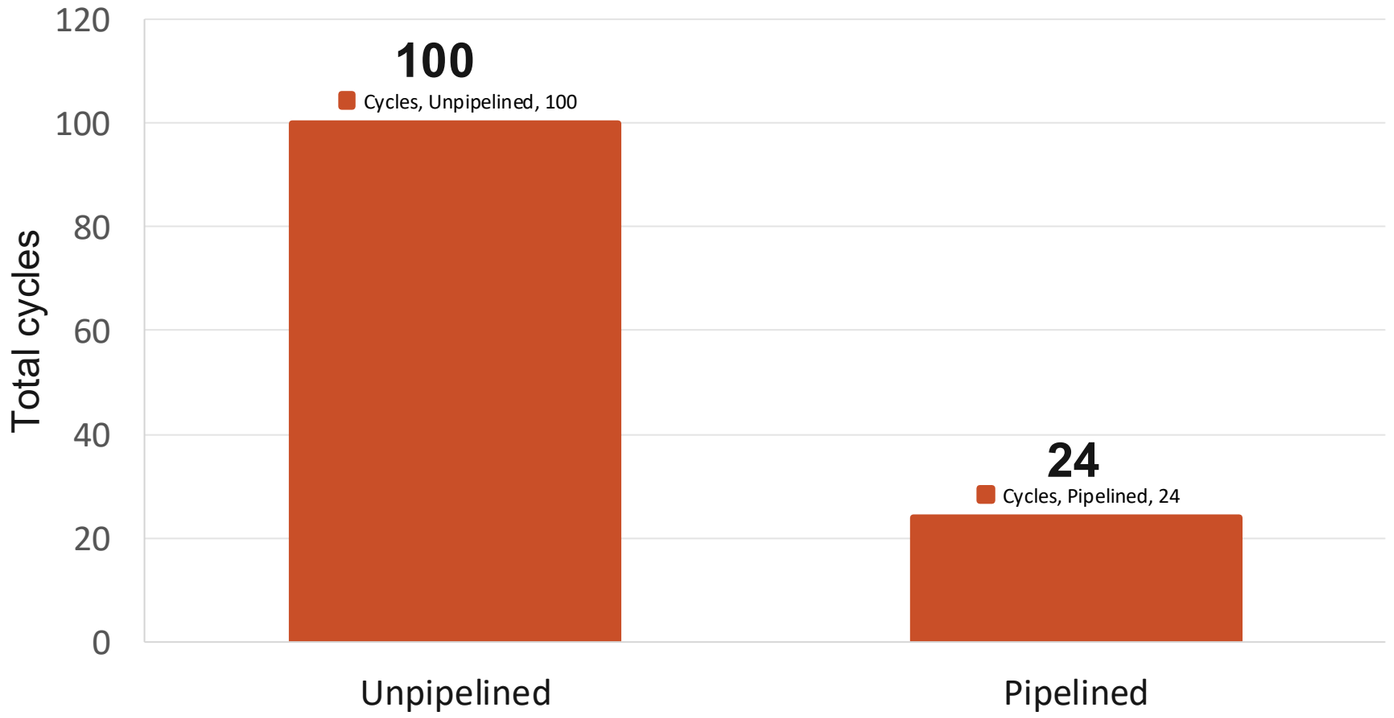
Instruction	1	2	3	4	5	6	7	8
I1	F	D	E	M	W			
I2		F	D	E	M	W		
I3			F	D	E	M	W	
I4				F	D	E	M	W

F Fetch D Decode E Execute M Memory W Write-back

Ideal steady-state throughput: one completed instruction per cycle.

WORKED EXAMPLE

A five-stage pipeline takes 24 cycles



20 instructions × 5 stages

Unpipelined: 20 × 5 = 100 cycles
Pipelined: 5 + 20 − 1 = 24 cycles

4.17×

ideal speedup for this example

Dependencies, branch mispredictions, and cache misses reduce this ideal gain.

Independent instructions can run together

Superscalar execution

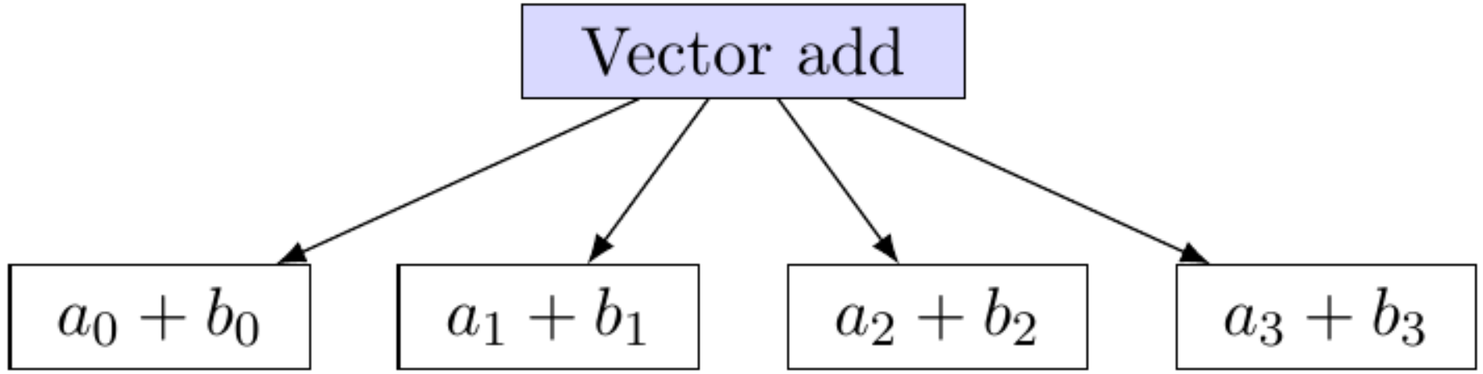
Issue multiple independent instructions in one cycle when execution units are available.

Out-of-order execution

Allow later independent work to proceed while an earlier operation waits.

Instruction sequence	Dependency
r1 = load A[i]	May wait for memory
r2 = b + c	Independent of r1; can proceed
r3 = r1 × d	Must wait for r1

SIMD applies one operation to many values



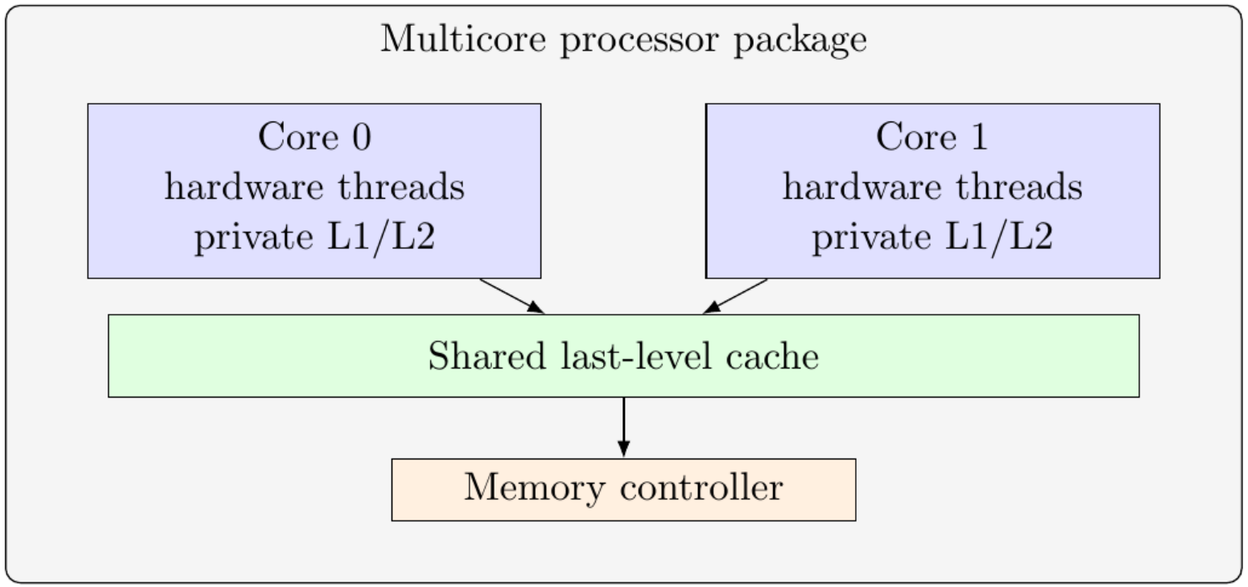
Good vectorization candidates

Independent iterations, contiguous data, suitable alignment, and little branching.

Quick calculation

Eight lanes × 12 vector additions = 96 scalar additions at maximum lane utilization.

Cores and hardware threads share resources



Multicore

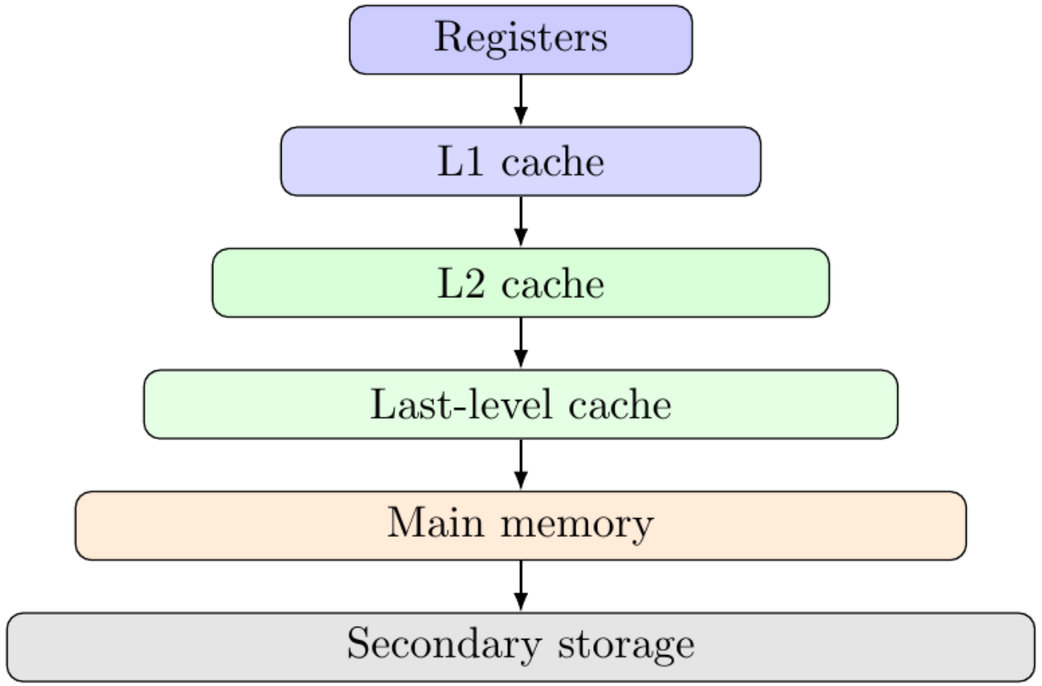
Physical cores execute different instruction streams.

Simultaneous multithreading

SMT keeps multiple hardware-thread states in one core and can use otherwise idle units.

More hardware threads do not imply an equal increase in compute capacity.

Keep frequently used data near the core



Small and fast near the core

Registers and caches serve frequent accesses quickly.

Larger capacity farther away

Main memory and storage hold more data, but access costs increase.

Locality reduces how often computation must cross a slower boundary.

Locality makes each fetched line useful

Kind of locality	Access pattern	Programming example
Spatial	Use nearby addresses soon	Traverse a row-major matrix by rows
Temporal	Reuse the same values soon	Reuse a tile before it leaves cache

Row-major traversal

A[0][0], A[0][1], A[0][2], …
Nearby elements share fetched lines.

Column traversal

A[0][0], A[1][0], A[2][0], …
A large stride can waste fetched bytes.

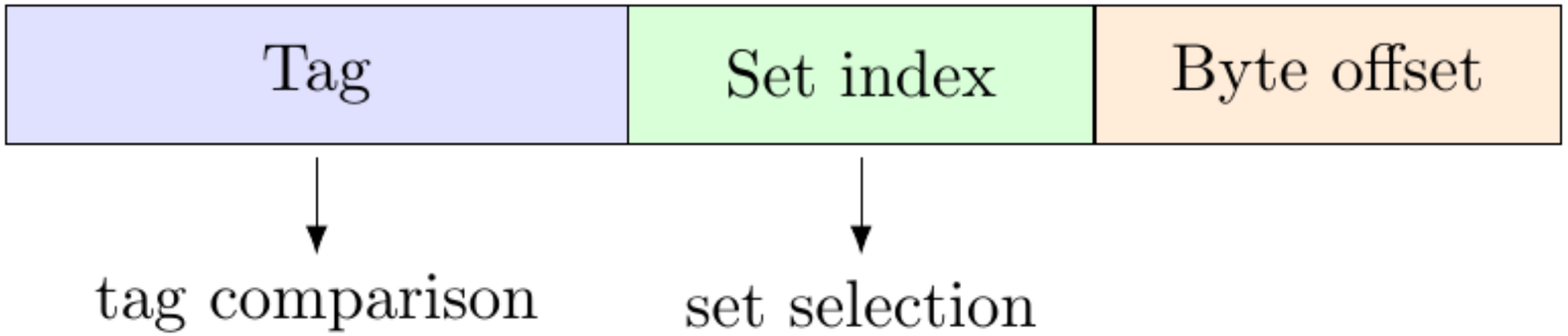
Identify why a cache access misses

Miss type	Cause	Possible response
Compulsory	First access to a cache line	Amortize the fetch through reuse
Capacity	Working set exceeds cache	Use smaller working tiles
Conflict	Active blocks map to the same set	Change layout or add padding
Coherence	Another core invalidates the line	Reduce sharing and ownership changes

Match the remedy to the cause of the miss.

WORKED EXAMPLE

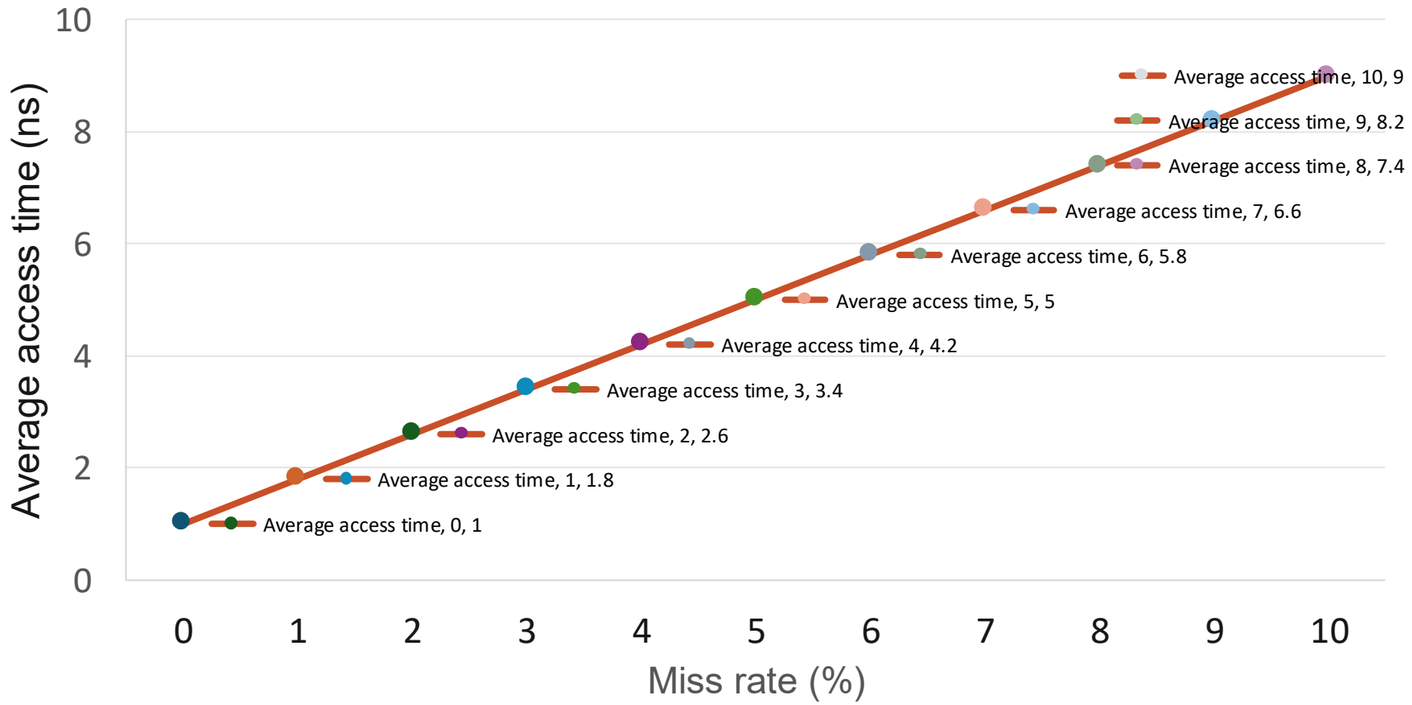
Sets, ways, and line size define capacity



64 sets × 4 ways × 64 bytes = 16 KiB

Byte offset	Set index	Tag
6 bits select a byte	6 bits select a set	Remaining upper address bits

A small miss rate can dominate access time



Tavg = Thit + rmiss × Tpenalty

Hit time: 1 ns
Additional miss penalty: 80 ns

5 ns

At a 5% miss rate:
1 + 0.05 × 80 = 5 ns

Calculated model: even 95% cache hits leave the average at five times the hit time.

Latency and bandwidth limit different work

Latency

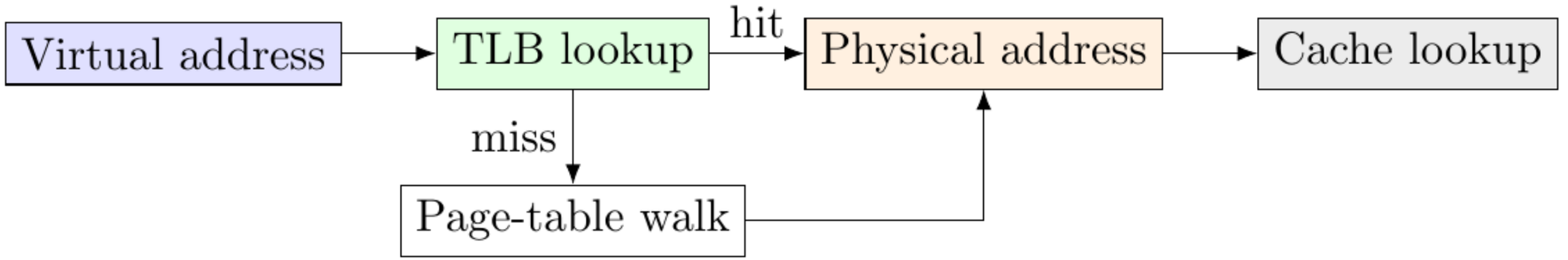
Time until an access or transfer begins to produce its result.

Bandwidth

Amount of data transferred per unit time.

Workload	Typical sensitivity	Why
Linked-list traversal	Latency	The next address depends on the last load
Streaming array addition	Bandwidth	Many independent elements need sustained traffic

The TLB caches address translations



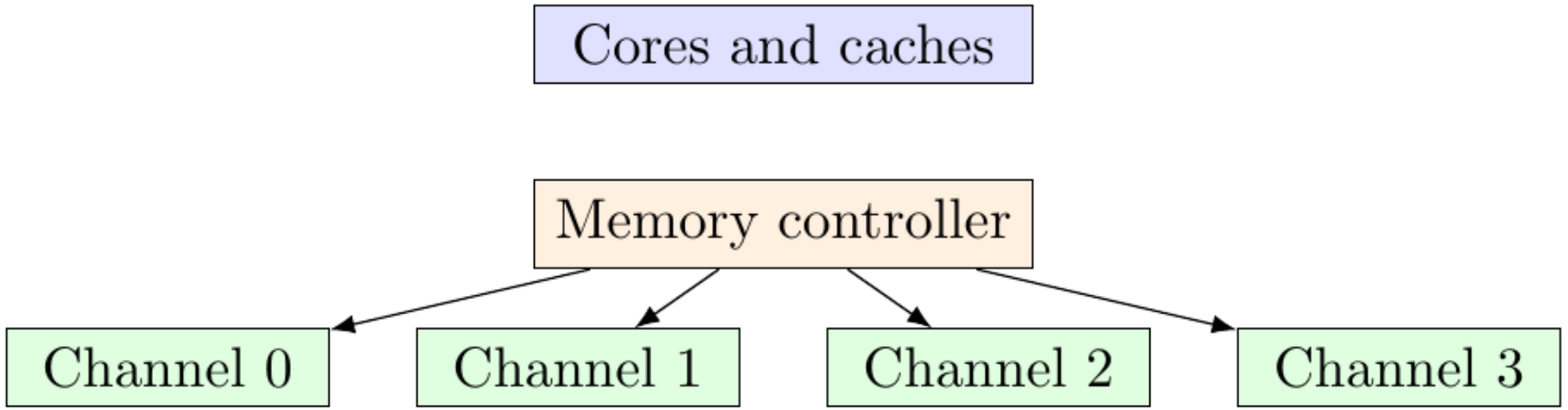
TLB miss

The translation is absent from the TLB. A page-table walk can find a resident page.

Page fault

The OS must handle the access; a nonresident page may need to be brought from storage.

Memory channels create parallel data paths



Interleaving

Spread consecutive blocks across channels or banks so independent requests can overlap.

Access pattern matters

A stride that repeatedly selects the same bank can serialize requests and leave others idle.

Memory traffic sets a throughput ceiling

Per iteration of c[i] = a[i] + b[i]	Bytes
Read a[i]	8
Read b[i]	8
Write c[i]	8
Total traffic	24

40 GB/s ÷ 24 B

≈ 1.67 billion

iterations per second

Bandwidth-only upper bound; assumes no write allocation or cache reuse.

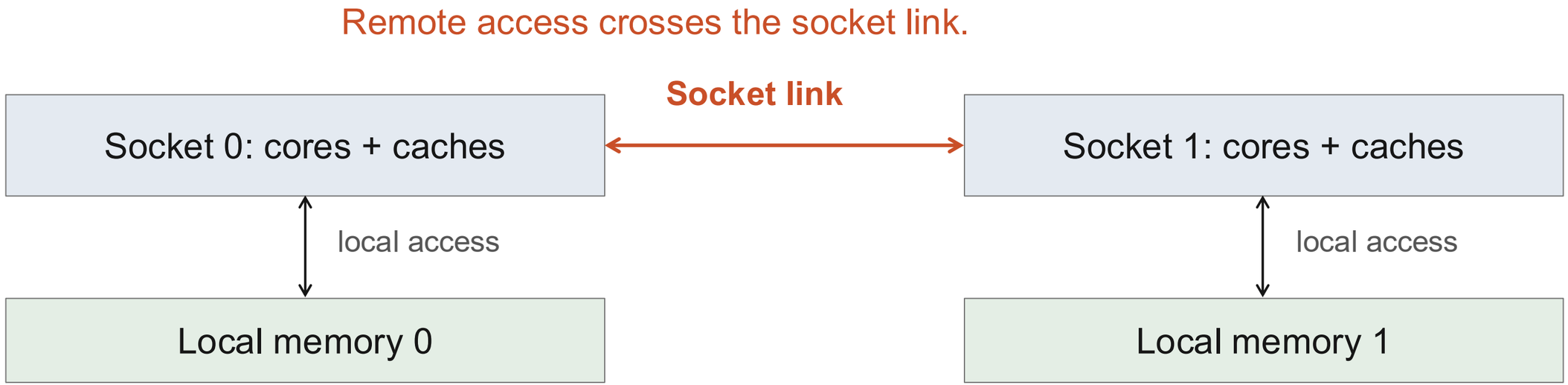
To raise this ceiling, reduce bytes moved per iteration or provide more bandwidth.

Shared memory may have unequal costs

Property	UMA	Cache-coherent NUMA
Address space	Shared	Shared
Main-memory cost	Approximately uniform	Local and remote costs differ
Memory connection	Similar access path for processors	Memory attached to different sockets
Main concern	Contention as the system grows	Thread affinity and data placement

Shared addressing makes access possible; placement determines its cost.

NUMA rewards matching threads to data



First-touch initialization

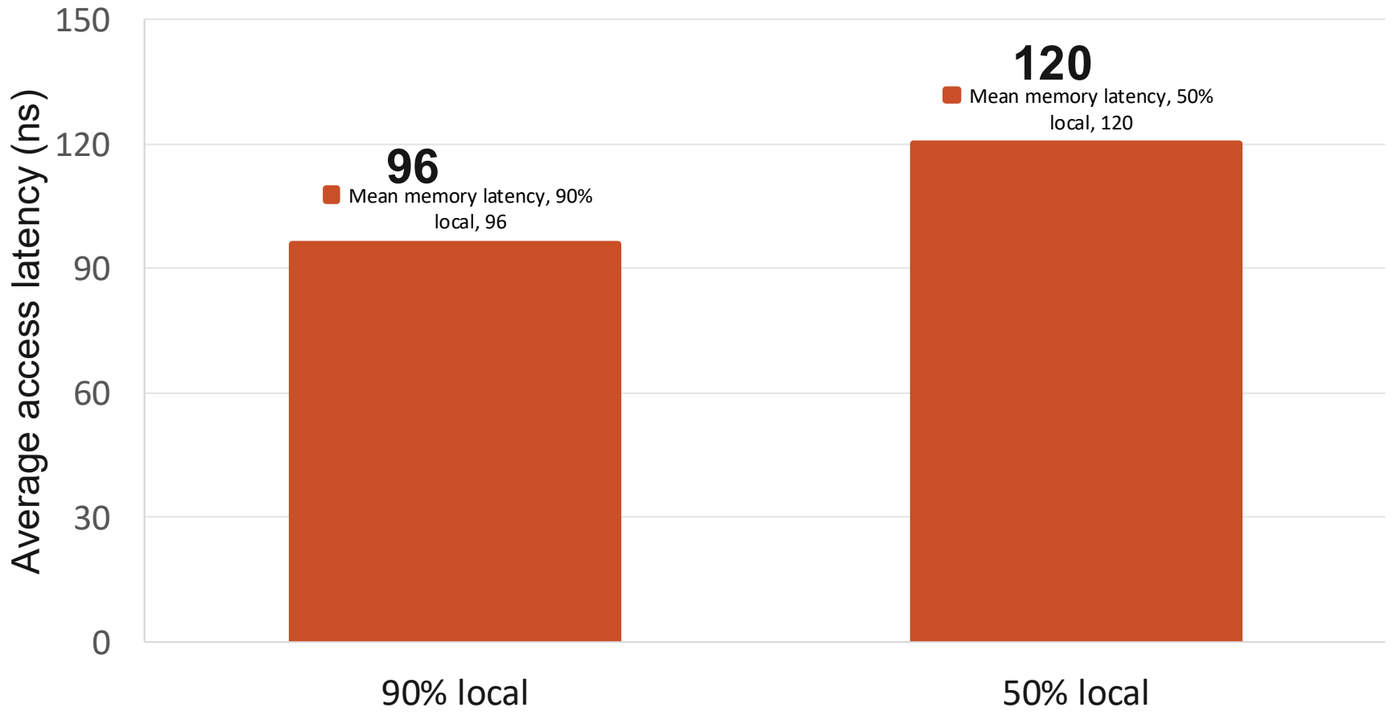
Initialize each data partition using the worker that will compute on it.

Thread affinity

Keep workers on chosen cores so their cache and NUMA locality remain stable.

WORKED EXAMPLE

Poor NUMA placement adds 25% latency



Local: 90 ns | Remote: 150 ns

90% local: 81 + 15 = 96 ns
50% local: 45 + 75 = 120 ns

+25%

(120 − 96) ÷ 96 × 100

Calculated access costs: processor count and arithmetic are unchanged.

Coherence coordinates copies of a line

Write-invalidate

A writer obtains exclusive ownership. Other cached copies are invalidated.

Write-update

A write sends the new value to other cached copies.

Step	Core 0	Core 1
Both read x	Cached copy of x	Cached copy of x
Core 0 writes x	Obtains ownership and updates x	Its cached line is invalidated
Core 1 reads again	May supply the current data	Obtains an up-to-date copy

MESI describes four cache-line states

State	Meaning	Key property
M Modified	Only valid cached copy	Differs from main memory
E Exclusive	Only clean cached copy	Can become Modified on a local write
S Shared	Clean copy; others may also hold it	Writing requires obtaining ownership
I Invalid	Local copy cannot be used	The next access must obtain the line

MESI state belongs to a cache line in a particular cache.

Follow ownership through a MESI sequence

Event	Core 0 state	Core 1 state
Initially: no cached copies	I	I
Core 0 reads the line	E	I
Core 1 reads the same line	S	S
Core 0 writes the line	M	I

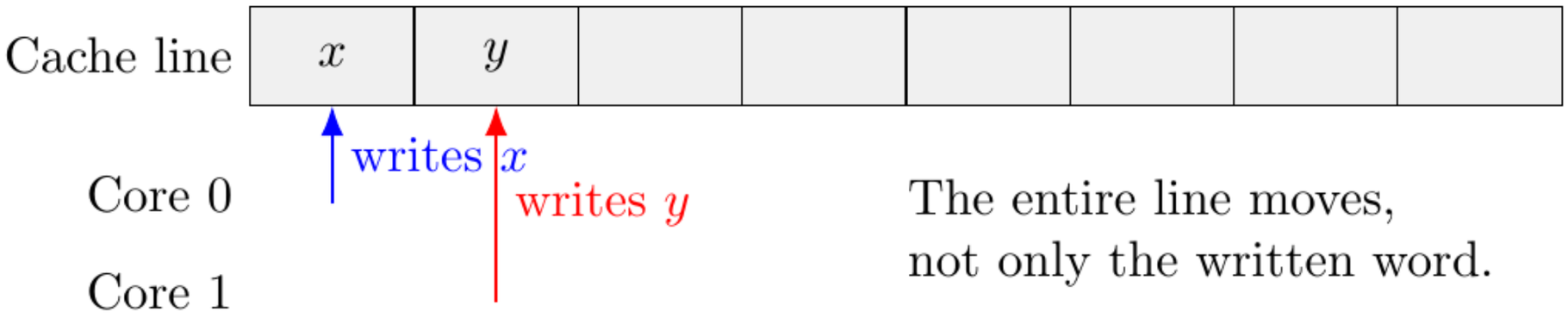
Snooping

Broadcast requests; caches watch for lines they hold.

Directory

Track sharers and target messages; add metadata and message costs.

False sharing moves an entire cache line



Different variables can still compete for ownership of the same line.

Remedy	Why it helps
Separate or align per-thread counters	Place frequent writes on different lines
Use private values or contiguous partitions	Give each thread clear ownership of writable data

Consistency constrains observation order

Coherence	Memory consistency
Coordinates values for one location	Constrains visible ordering across operations
Keeps cached copies coherent	Determines which reorderings are allowed

Producer

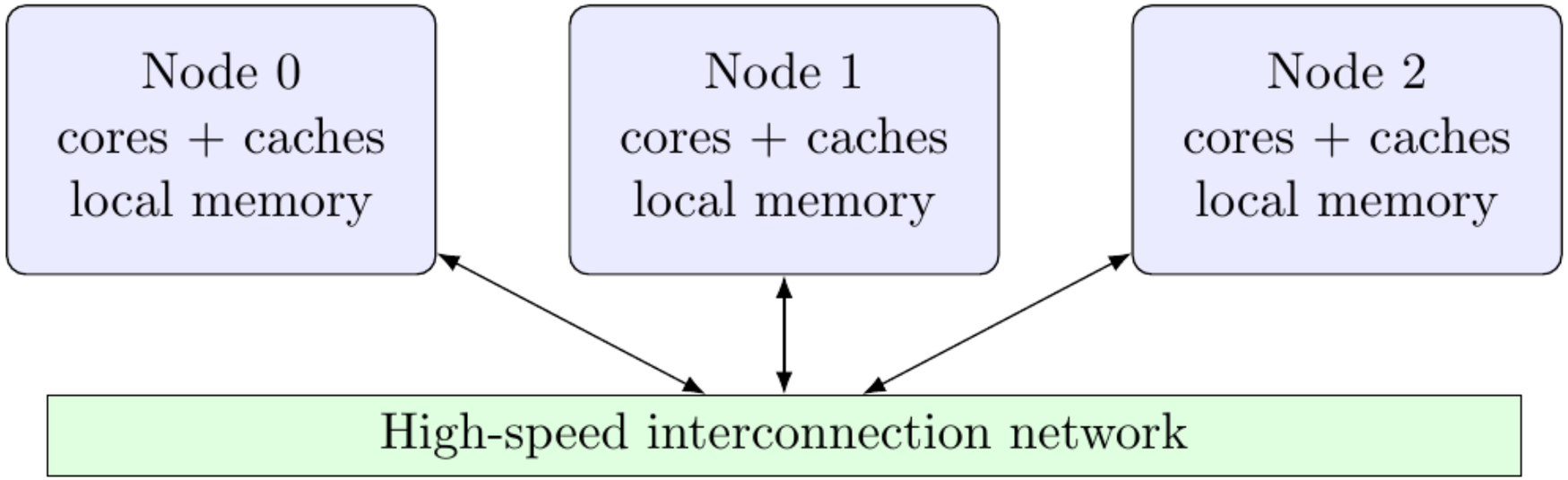
1. Write the shared payload.
2. Publish ready using a release operation.

Consumer

1. Observe ready with matching acquire.
2. Read the published payload.

Locks, atomics, and appropriate fences provide ordering; timing guesses do not.

Clusters connect separate memory spaces



Capacity and scale

Distribute the problem across nodes when one machine is too small.

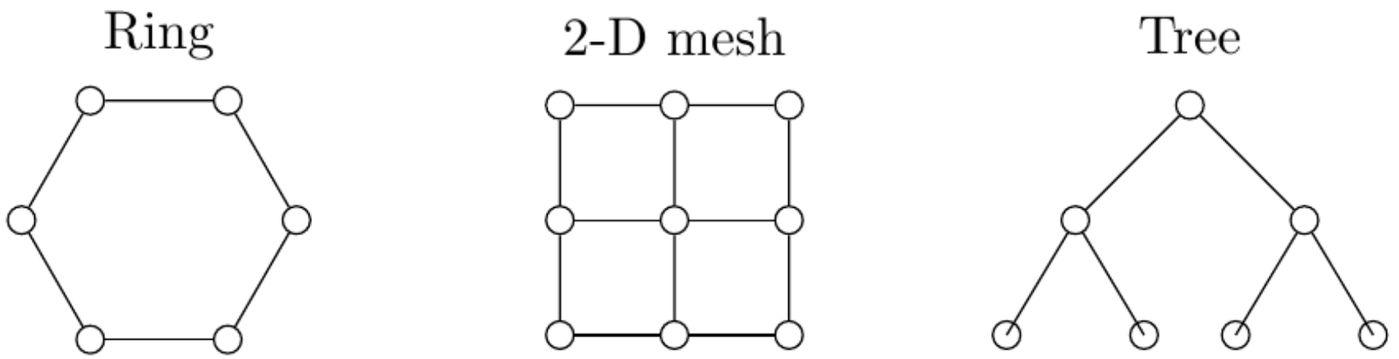
Communication cost

Choose data partitions carefully and exchange only the data needed.

Evaluate a network beyond link speed

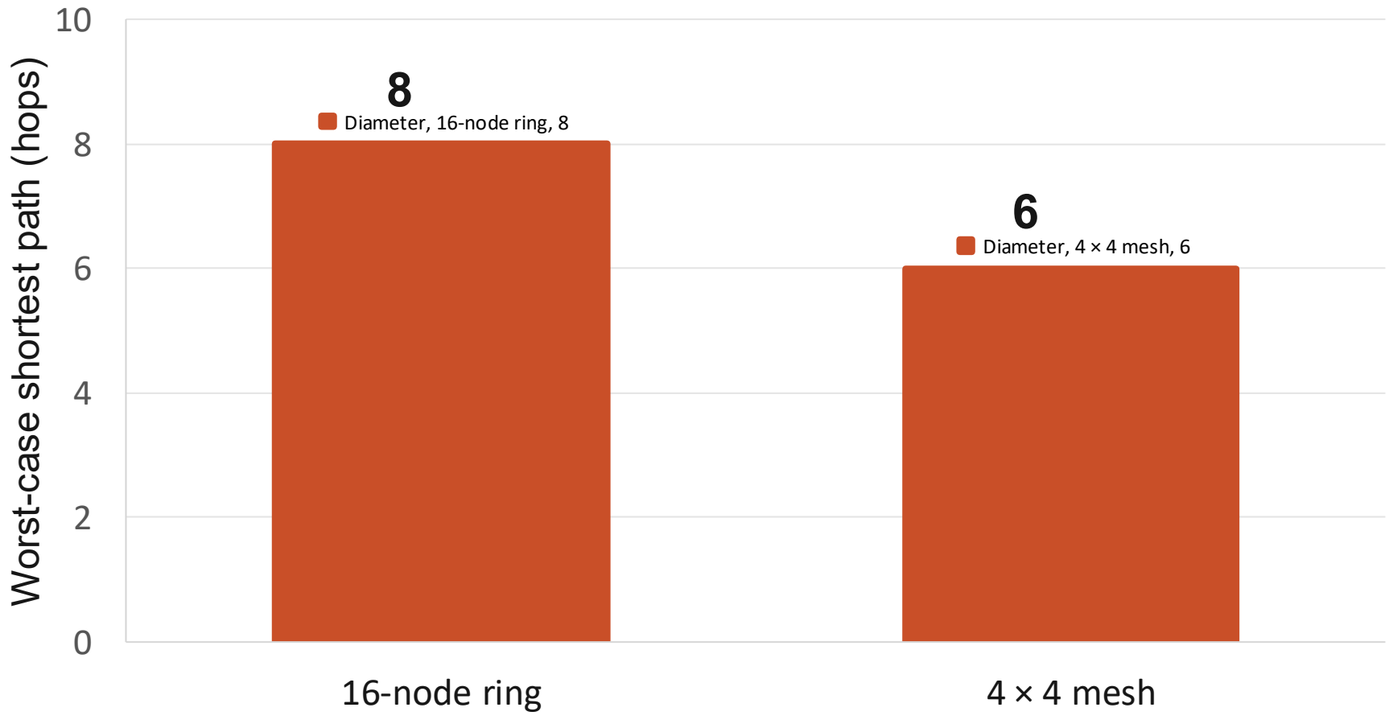
Metric	What it tells you
Node degree	How many direct links a node has
Diameter	Largest shortest-path hop count
Latency	Startup and propagation delay
Bandwidth	Sustained data volume per second
Bisection bandwidth	Capacity across a cut dividing the network into similar halves
Contention	Delay when transfers compete for links

Topology trades wiring for communication



Topology	Strength	Main limitation
Bus / crossbar	Simple bus / concurrent crossbar paths	Shared bus bottleneck / crossbar cost
Ring	Regular layout with modest wiring	Hop count grows with network size
Mesh / torus	Scalable local wiring	Traffic crosses multiple hops
Tree / fat tree	Hierarchy / higher aggregate capacity	Root bottlenecks / expensive upper links

A mesh lowers the worst-case hop count



Ring

Diameter = floor(N / 2)
For N = 16: 8 hops, 16 links.

Two-dimensional mesh

Diameter = (r − 1) + (c − 1)
For 4 × 4: 6 hops, 24 links.

The mesh reduces the diameter here, but uses more physical links.

Shared links turn traffic into contention

Mechanism	Benefit	Tradeoff or risk
Deterministic routing	Simple, predictable paths	A fixed path may be congested
Adaptive routing	Can avoid congested links	Requires more complex decisions
Flow control and buffers	Manage available transfer capacity	Insufficient space can stall progress

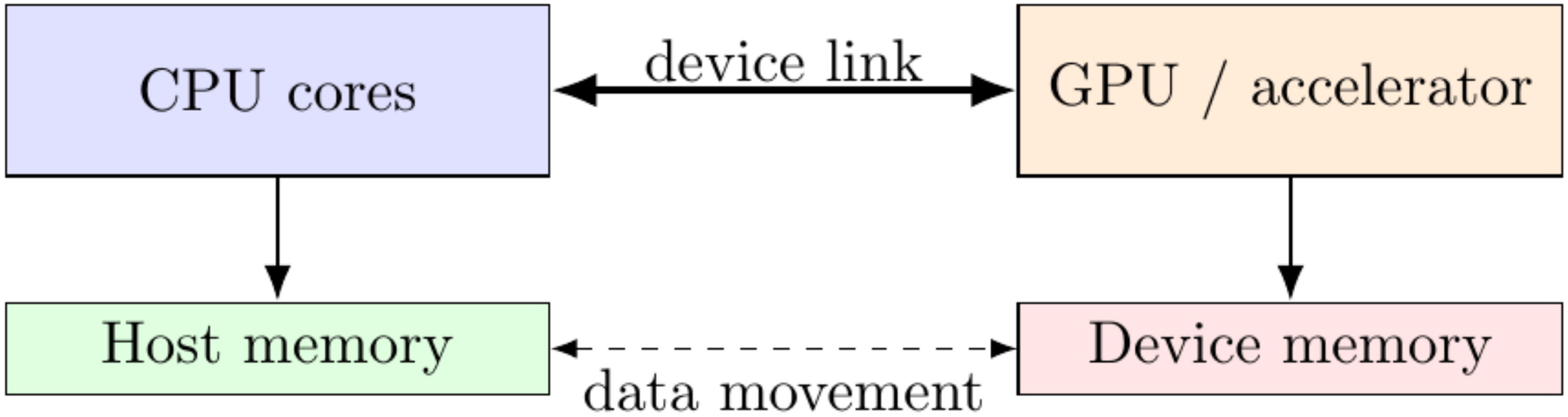
Head-of-line blocking

One delayed packet prevents unrelated packets behind it from advancing.

Inside a processor too

Core traffic competes for shared paths even without shared application variables.

Accelerators add a data-movement boundary



Choose suitable work

Use the CPU for control-heavy work and the accelerator for regular, reusable kernels.

Keep data close

Reuse data on the device so useful computation justifies transfer costs.

Unified memory can simplify addressing; data movement still has a cost.

Matrix multiplication uses every level

Level	How tiled C = AB benefits
Instruction + SIMD	Overlap multiply-add work and update several elements per instruction
Cache	Reuse blocks of A and B before eviction
Multicore + NUMA	Assign output tiles to threads and place pages near their socket
GPU	Use many threads and reuse input tiles in on-device shared memory
Cluster	Distribute matrix blocks and exchange needed panels

The arithmetic stays the same; ownership and data movement change.

Choose hardware around the workload

Workload	Suitable starting point	Placement priority
Latency-sensitive web service	CPU-based shared-memory server	Keep hot data local; limit contention
Dense neural-network training	GPU-equipped system	Reuse tensors on the accelerator
Weather simulation beyond one node	Distributed-memory cluster	Partition the domain; exchange boundaries

Before deciding, ask:

How much independent work? • How large is the working set?
How much communication? • Can computation reuse data where it lives?

PRACTICE

Check your architectural reasoning

1 Average access time

Hit time 2 ns, miss rate 4%, extra miss cost 100 ns. What is the average?

2 Cache organization

128 sets, 8 ways, 64-byte lines. Find capacity and index/offset bit counts.

3 NUMA placement

Local 80 ns; remote 140 ns. Compare average latency at 75% and 95% local access.

4 Independent counters

Two threads update different counters in one line. Explain the cost and propose two fixes.

CHAPTER SYNTHESIS

Place work and data to reduce movement

Expose useful parallelism

Use independent instructions, SIMD lanes, cores, and nodes when the workload supports them.

Control memory costs

Exploit locality, reduce misses, respect bandwidth limits, and place NUMA pages deliberately.

Coordinate shared state

Understand coherence and ordering; synchronize correctly and avoid false sharing.

Match the architecture

Choose CPU, GPU, or cluster resources using access patterns, capacity, and communication cost.

Apply it: name a workload, identify its likely bottleneck, and justify one placement choice.